

Exercícios — Aula 01

Curso de Formação de Programação Aplicada · Módulo 1 · FPA Labs

Como fazer: crie uma pasta `minhas-solucoes/` aqui dentro e resolva cada exercício em um arquivo separado. Só olhe a pasta `solucoes/` **depois** de tentar de verdade — inclusive depois de errar algumas vezes. Errar e ler a mensagem de erro é metade do aprendizado.

Nível 1 — obrigatório

Exercício 1 — Cartão de visitas

Crie o arquivo `01-cartao.js` que imprime no terminal seus dados, assim:

```
=====
Maria Silva
Desenvolvedora em formação
maria@email.com
=====
```

Requisitos:

- Guarde nome, cargo e e-mail em `const` separadas.
- Use template literals (crases) para montar as linhas.

Exercício 2 — Calculadora de linha de comando

Crie `02-calculadora.js` que recebe dois números pelo terminal e mostra as quatro operações:

```
node 02-calculadora.js 10 3
```

```
10 + 3 = 13
10 - 3 = 7
10 * 3 = 30
10 / 3 = 3.3333333333333335
```

Requisitos:

- Use `process.argv` para ler os números.
- ⚠ `process.argv` sempre devolve **texto**. `"10" + "3"` dá `"103"`, não `13`. Converta com `Number(...)`.

- Se faltar algum número, mostre `Uso: node 02-calculadora.js <n1> <n2>` e encerre.

Exercício 3 — Projeto com módulos

Crie uma pasta `03-conversor/` com um projeto completo:

```
03-conversor/  
├─ package.json      (npm init -y, e adicione "type": "module")  
├─ conversoes.js  
└─ index.js
```

Em `conversoes.js`, exporte três funções:

Função	O que faz
<code>celsiusParaFahrenheit(c)</code>	<code>c * 9/5 + 32</code>
<code>kmParaMilhas(km)</code>	<code>km * 0.621371</code>
<code>reaisParaDolares(reais, cotacao)</code>	<code>reais / cotacao</code>

Em `index.js`, importe as três e mostre o resultado de cada uma.

Faça funcionar com `npm start`.

Nível 2 — esperado

Exercício 4 — Conversor pelo terminal

Melhore o exercício 3: em vez de valores fixos no código, receba pelo terminal.

```
npm start -- temperatura 25  
node index.js distancia 42  
node index.js moeda 100 5.4
```

Dicas:

- `process.argv[2]` é a operação; do `[3]` em diante são os valores.
- Use `if/else` ou `switch` para escolher a função.
- Se a operação não existir, mostre a lista de operações válidas.

Exercício 5 — Formatando a saída com um pacote

Instale o `chalk` no projeto do exercício 4:

```
npm install chalk
```

Deixe a saída colorida:

- resultado em **verde**
- mensagens de erro em **vermelho**
- o título em **negrito azul**

Depois, abra o `package.json` e confira: o `chalk` apareceu em `dependencies`? Qual versão? Existe um `package-lock.json`?

Desafio — opcional

Exercício 6 — Mini agenda no terminal

Crie um projeto `agenda/` com:

- `contatos.js` — exporta um array de contatos (nome, telefone, e-mail) e as funções `listarTodos()`, `buscarPorNome(nome)` e `contarContatos()`.
- `index.js` — usa `process.argv` para decidir o que fazer:

```
node index.js listar
node index.js buscar Maria
node index.js contar
```

Requisitos:

- A busca deve ignorar maiúsculas/minúsculas (dica: `.toLowerCase()`).
- Se não encontrar ninguém, mostre `Nenhum contato encontrado com "xyz"`.
- A listagem deve sair formatada e legível.

Se quiser ir além: faça `node index.js` sem argumentos mostrar um menu de ajuda com todos os comandos disponíveis.

Checklist de entrega

- ☐ Todos os arquivos rodam sem erro
- ☐ Nenhum `node_modules/` foi enviado junto
- ☐ Os projetos com `import` têm `"type": "module"` no `package.json`
- ☐ O código está indentado e as variáveis têm nomes que explicam o que guardam

